Common Core: Phase 2 Verification

Lab V1: Self-Checking Testbench and Assertions

Randomization, Plusargs, Regression and SVA for seq_mul and sync_fifo

ASIC Design, Verification and Implementation Onboarding

Kerim Turak

October 24, 2026

LAB OBJECTIVE Turn the designer's smoke testbenches of `seq_mul` and `sync_fifo` into self-checking verification environments with a driver, monitors, a reference model and a scoreboard; drive them with reproducible random stimulus configured by plusargs; run them as a regression; and add SystemVerilog assertions and cover properties that point at a bug in the cycle where it happens.

Duration	2 weeks (approximately 30–40 hours)
Prerequisites	Labs L0, R1 and R2
Designs	`seq_mul`, `sync_fifo` (verified, RTL unchanged unless a bug is found); `tb_utils_pkg` (new, shared)
Reused in	V2 and V3 (`dv/` structure, `tb_utils_pkg`, `regress`), P1 (gate-level simulation runs these testbenches), T-RTL3 (the `seq_mul` environment verifies the pipelined multiplier)

Contents

1 Overview **3**

2 From Smoke Test to Verification Environment **3**

2.1 The `dv/` Directory . . . 3

2.2 The Testplan . . . 4

3 Self-Checking Testbench Architecture **4**

3.1 Required Structure . . . 5

3.2 Driving and Sampling . . . 6

3.3 Skeleton . . . 6

3.4 The `sync_fifo` Environment . . . 8

4 Stimulus, Plusargs and Reproducibility **8**

4.1 Compile-Time Parameters Versus Runtime Plusargs . . . 8

4.2 Randomization Functions . . . 9

4.3 Reproducibility . . . 10

4.4 Shared Utilities: `tb_utils_pkg` . . . 11

5 Required Tests **12**
5.1 `seq_mul` . . . 12
5.2 `sync_fifo` . . . 12

6 Regression **12**
6.1 The Regression List . . . 13
6.2 `flow/mk/regress.mk` . . . 13

7 Assertions (SVA) **14**
7.1 Immediate and Concurrent Assertions . . . 14
7.2 Sampling: Why Assertions Do Not Race . . . 14
7.3 Implication and Delay . . . 15
7.4 Sampled-Value Functions . . . 15
7.5 Assert, Assume and Cover . . . 16
7.6 Where Assertions Go . . . 16
7.7 Required Assertions . . . 18
7.8 Exercise: See an Assertion Fire First . . . 18

8 Requirements / Tasks **19**

9 Repository and Flow **19**

10 Deliverables **20**

11 Assessment **21**

12 Review Questions **21**

13 References **21**

1 Overview

In Phase 1 you designed `seq_mul` (Lab R1) and `sync_fifo` (Lab R2) and used their directed tests to check basic behavior. Those tests do not answer the question a project lead asks before tape-out: "how do we know it works in every situation the rest of the chip can create?".

In a real team the answer comes from a verification environment that is separate from the design, written against the *specification* rather than against the RTL, and run thousands of times with different random stimulus every night. Its results are judged by three independent kinds of evidence: a scoreboard that compares every output with a reference model, assertions that check rules on every clock edge, and cover properties that prove the interesting scenarios were actually reached. This lab builds the first, simple version of all three on the two smallest designs of the program, so that the ideas are clear before the designs become harder.

The environment is built from tasks and functions, without classes. The separation you build here — drive, observe, predict, compare — is the driver / monitor / reference model / scoreboard structure that Lab V3 expresses with classes and Lab T-DV2 with UVM.

By the end of this lab you should be able to:

1. Structure a verification environment under `hw/ip/<ip>/dv/` with a driver, monitors, a reference model and a scoreboard that each have one job.
2. Generate random stimulus with `$urandom` and `$urandom_range`, avoid the classic pitfalls, and reproduce any failing run from its seed.
3. Configure tests at run time with plusargs and at compile time with parameters, and explain why the two are different.
4. Run a regression of tests × seeds × parameter sets with one command and read its summary.
5. Write immediate and concurrent assertions with implication, delays and the sampled-value functions `$rose`, `$fell`, `$stable` and `$past`, and attach them to a design with `bind`.
6. Use cover properties to show that assertions were not only passing vacuously.
7. Write a testplan that maps every feature of a design to a test, an assertion or a cover item.

Where this lab sits. It builds on the RTL of Labs R1 and R2, on the scheduling rules and the `<=` drive standard from Lab R2, and on the flow skeleton from Lab L0 (`PARAMS`, `DEFINES`, `TEST`, `SEED`, `PLUSARGS`, `WAVES`). Its output is reused directly: Lab V2 applies the same structure to `arb_mem`, Lab V3 rebuilds it with classes, Lab P1 runs these testbenches on the gate-level netlists, and Lab T-RTL3 verifies the pipelined multiplier with the `seq_mul` environment unchanged.

2 From Smoke Test to Verification Environment

2.1 The `dv/` Directory

The smoke testbench in `hw/ip/<ip>/tb/` stays where it is and keeps passing: it is the designer's quick check and it is part of `make check-all`. The verification environment is new and lives next to it, following the OpenTitan layout:

```
hw/ip/sync_fifo/dv/
+-- testplan.md            features -> tests / assertions / cover items
+-- regress.list           configurations x tests x seeds for `make regress`
+-- dv.f                   testbench file list (TB_FLIST)
+-- tb/tb_top.sv           clock, reset, DUT, plusargs, test start, watchdog
+-- env/
|   +-- sync_fifo_driver.svh       tasks that drive DUT inputs
```

```
|   +-- sync_fifo_monitor.svh      tasks that observe handshakes
|   +-- sync_fifo_scoreboard.svh   reference model + compare
+-- tests/sync_fifo_tests.svh      one task per +TEST= name, run_test()
+-- sva/
    +-- sync_fifo_sva.sv           assertion checker module
    +-- sync_fifo_bind.sv          bind sync_fifo sync_fifo_sva ...
```

The same structure exists for `seq_mul`. Shared, design-independent code (report counters, `TEST PASSED`/`TEST FAILED`, watchdog, plusarg helpers) goes once into `dv/common/tb_utils_pkg.sv`.

The driver, monitor and scoreboard files are `.svh` files `` `included `` into `tb_top`. They contain tasks and functions that read and write the DUT signals declared in `tb_top`, so they must be compiled inside its scope. Keeping them in separate files still gives each component one owner and one place to change. In Lab V2 the tasks receive the DUT through a *virtual interface* argument instead of module scope, and in Lab V3 they become classes.

2.2 The Testplan

A verification environment without a plan grows by accident: tests are added where bugs happened to be found, and nobody can say what is *not* checked. Industrial flows therefore start from a testplan. OpenTitan keeps one per block (`hw/ip/<ip>/data/<ip>_testplan.hjson`) that lists every feature with the tests that exercise it; its regression tool reads the file and reports progress per feature. You will use the same idea in a lighter form: a Markdown table in `dv/testplan.md`.

```
# sync_fifo testplan  (dv/testplan.md)
| Feature       | Description                     | Tests        | Assertions         | Covers       |
|---------------|---------------------------------|--------------|--------------------|--------------|
| fwft          | pop_data shows oldest entry     | smoke,random | a_pop_data_stable  | -            |
| push_full     | push while full is ignored      | fill_drain   | a_count_step       | c_push_full  |
| push_pop_full | push+pop while full: both occur | simultaneous | a_count_step       | c_pp_full    |
| flags         | full/empty agree with count     | all          | a_full_def         | c_full,c_f2e |
| odd_depth     | pointer wrap, DEPTH=5           | random@w32d5 | a_count_max        | c_wrap       |
```

Write the testplan *first*, from the specifications in `doc/spec.md`, before writing tests. Every row must end up with at least one test that passes and, where it makes sense, an assertion and a cover item. A feature with an empty row is a known gap and is listed in `doc/verification.md`. Lab V3 extends the testplan with functional coverage items.

3 Self-Checking Testbench Architecture

Figure 1 shows the environment of this lab for `seq_mul`; the `sync_fifo` environment has the same structure (one monitor, a queue as model).

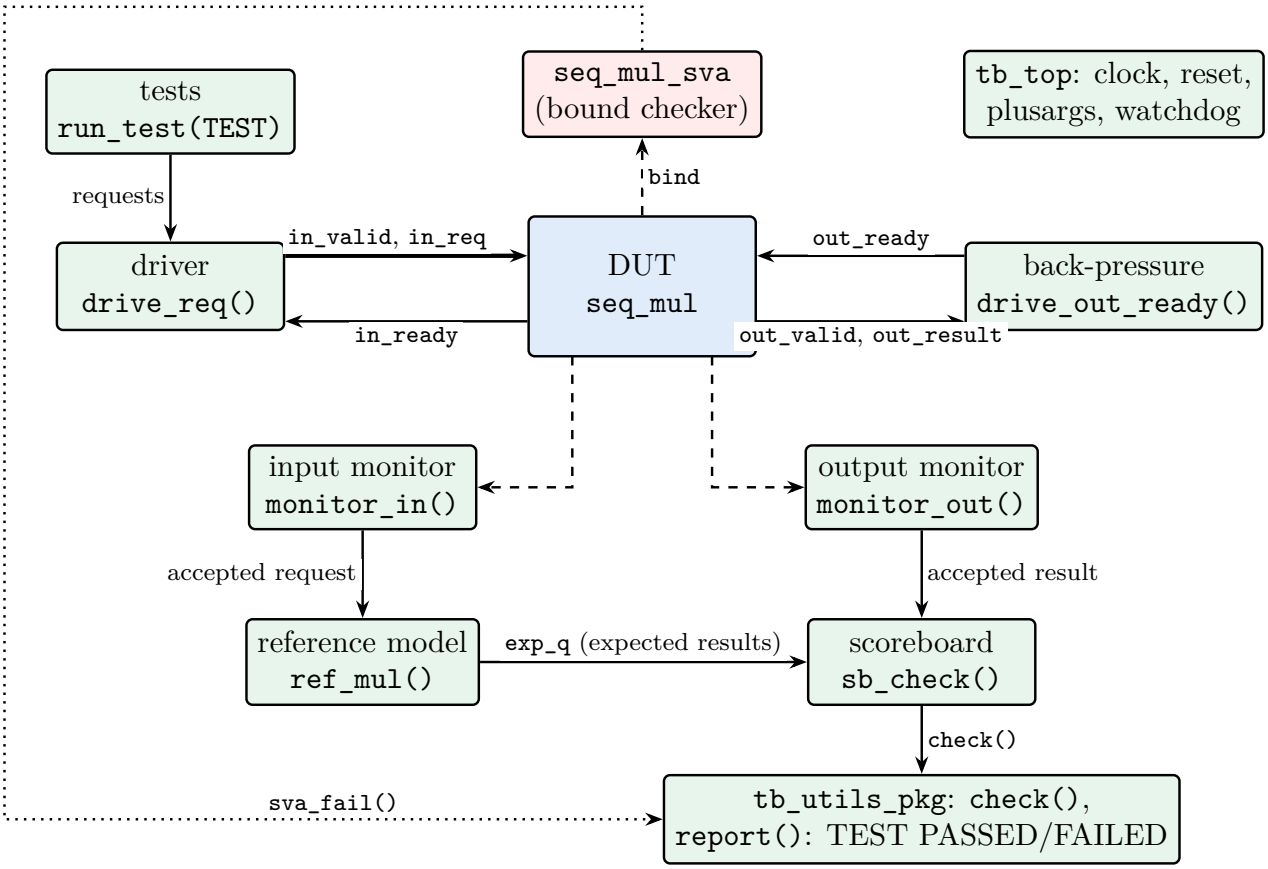


Figure 1: Self-checking environment of Lab V1 (green: testbench, blue: DUT). The driver drives with `<=`; the monitors only observe handshakes; the reference model fills the expected queue `exp_q` that the scoreboard pops; the bound SVA checker and the scoreboard both report through `tb_utils_pkg`, which prints the single `TEST PASSED`/`TEST FAILED` line.

3.1 Required Structure

Component	Implemented as (`seq_mul`)	Responsibility
Reset	`task automatic apply_reset()`	Assert `rst_n` for a few cycles, then release it synchronously
Driver	`task automatic drive_req(input mul_req_t req)`	Drive one request with `<=` and wait until the DUT accepts it
Input monitor	`task automatic monitor_in()`	Watch the input handshake; on every accepted request, call the reference model and push the expected result into a queue
Reference model	`function automatic logic [XLEN-1:0] ref_mul(mul_req_t req)`	Compute the correct result in a simple, obviously correct way
Output monitor	`task automatic monitor_out()`	Watch the output handshake; pass every accepted result to the scoreboard
Scoreboard	`function automatic void sb_check(logic [XLEN-1:0] got)`	Pop the expected value and compare; an empty queue is an error (unexpected output)
Back-pressure	`task automatic drive_out_ready()`	Toggle `out_ready` randomly (`BP_PCT`) to test that the result is held stable
Report	`tb_utils_pkg::report()`	Print the number of checks and errors; print `TEST PASSED` or `TEST FAILED`

Rules of the structure:

- Monitors never drive DUT inputs, and the driver never checks results. A component with two jobs cannot be reused for the other job.
- Monitors identify transactions only through the handshake (`valid && ready` at a rising edge), never by counting cycles. The testbench must not assume a fixed latency; the specification does not promise one, and Lab T-RTL3 will change it.
- The reference model is written from the specification, not from the RTL. Copying the RTL algorithm into the model copies its bugs.
- At the end of the test the expected queue must be empty. A result that was expected but never came is an error, exactly like a wrong result.

TEST TERM | Back-pressure test

The consumer randomly de-asserts its ready signal (`out_ready`, or the FIFO's `pop`) with probability `BP_PCT`, so the DUT must hold its result and the testbench must cope with stalls. A pass proves the handshake works in every stall pattern the seeds produced, not only at full rate; most interface bugs (a result dropped or changed while waiting) appear only here.

3.2 Driving and Sampling

All DUT inputs are driven with `<=` synchronous to the rising edge, the standard from Lab R2. The process wakes up at the edge in the Active region, reads DUT outputs before any flip-flop has been updated, and schedules its own changes in the NBA region, after the DUT has sampled its inputs. The DUT therefore sees each change one edge later, exactly as if it came from another flip-flop. Monitors sample at the same rising edge and see the values the DUT sampled.

3.3 Skeleton

The skeleton shows the structure only; complete the TODOs and split it into the files of the `dv/` directory.

```
module tb_top;
  import mul_pkg::*;
  import tb_utils_pkg::*;

  logic clk = 1'b0;
  logic rst_n;
  logic in_valid, in_ready, out_valid, out_ready;
  mul_req_t in_req;
  logic [XLEN-1:0] out_result;

  logic [XLEN-1:0] exp_q[$];          // expected results, in order
  int unsigned bp_pct  = 20;          // plusarg +BP_PCT=
  int unsigned num_txn = 10000;       // plusarg +NUM_TXN=

  initial forever #5 clk = ~clk;

  seq_mul dut (.*);

  // ---------------- reference model (env/seq_mul_scoreboard.svh) --------
  function automatic logic [XLEN-1:0] ref_mul(mul_req_t r);
    logic [2*XLEN-1:0] p;
    unique case (r.op)
      OP_MUL, OP_MULHU:
```

```
      p = {{XLEN{1'b0}}, r.a} * {{XLEN{1'b0}}, r.b};
    OP_MULH:
      p = $signed({{XLEN{r.a[XLEN-1]}}, r.a}) *
          $signed({{XLEN{r.b[XLEN-1]}}, r.b});
    OP_MULHSU:
      p = $signed({{XLEN{r.a[XLEN-1]}}, r.a}) *
          $signed({{XLEN{1'b0}}, r.b});
    default: p = 'x;
  endcase
  return (r.op == OP_MUL) ? p[XLEN-1:0] : p[2*XLEN-1:XLEN];
endfunction

function automatic void sb_check(logic [XLEN-1:0] got);
  logic [XLEN-1:0] exp;
  if (exp_q.size() == 0) begin
    check(1'b0, $sformatf("unexpected result %h", got));
    return;
  end
  exp = exp_q.pop_front();
  check(got === exp, $sformatf("result %h, expected %h", got, exp));
endfunction

// ---------------- driver (env/seq_mul_driver.svh) ----------------
// Precondition: called right after a rising edge.
task automatic drive_req(input mul_req_t req);
  in_req   <= req;
  in_valid <= 1'b1;
  do @(posedge clk); while (!in_ready);   // accepted at this edge
  in_valid <= 1'b0;
endtask

task automatic drive_out_ready();
  forever begin
    @(posedge clk);
    out_ready <= ($urandom_range(99) >= bp_pct);
  end
endtask

// ---------------- monitors (env/seq_mul_monitor.svh) ----------------
task automatic monitor_in();
  forever begin
    @(posedge clk);
    if (rst_n && in_valid && in_ready) exp_q.push_back(ref_mul(in_req));
  end
endtask

task automatic monitor_out();
  forever begin
    @(posedge clk);
    if (rst_n && out_valid && out_ready) sb_check(out_result);
  end
endtask

`include "seq_mul_tests.svh"        // run_test(name): one task per test

initial begin
  in_valid  = 1'b0;
  out_ready = 1'b1;
  read_config();                     // plusargs, prints CONFIG line
  fork
```

```
      watchdog(100ms);                  // tb_utils_pkg
    join_none
    apply_reset();
    fork
      monitor_in();
      monitor_out();
      drive_out_ready();
    join_none
    run_test(test_name);                // TODO: unknown name -> error
    // TODO: wait until exp_q is empty (bounded), then:
    report();                           // TEST PASSED / TEST FAILED
    $finish;
  end
endmodule
```

The testbench uses `===` and `!==` on purpose: a result that contains X must be reported as a failure. (The rule "never `===` in RTL" from Lab R1 applies to RTL only.)

3.4 The `sync_fifo` Environment

The FIFO environment has the same components with FIFO-specific contents:

- **Driver:** each cycle drives `push`, `push_data` and `pop` with `<=`, chosen by the test (weighted random in `random`). The driver may push while `full` and pop while `empty` on purpose: the specification says those requests are ignored, and that must be checked.
- **Monitor and reference model:** at each rising edge the monitor computes which operations were *accepted* (`push && (!full || pop)`, `pop && !empty`) and applies them to the model, a queue `logic [WIDTH-1:0] model_q[$]`. The smoke test of Lab R2 already used such a queue; here it becomes the scoreboard's reference.
- **Scoreboard:** on every accepted pop, `pop_data` must equal `model_q[0]` before the pop is applied. On every cycle, `count`, `full` and `empty` must match the model (`model_q.size()`).

Be careful with the order inside one edge: compare first, then update the model with this edge's accepted operations, exactly as the DUT's flip-flops do.

4 Stimulus, Plusargs and Reproducibility

4.1 Compile-Time Parameters Versus Runtime Plusargs

Lab L0 introduced the two mechanisms; this lab uses both heavily. Keep them apart:

	Parameters / defines	**Plusargs**
When fixed	At compile (elaboration) time	At run time
Changes hardware?	Yes: widths, depths, number of flops	No: only testbench behavior
Passed with	`PARAMS=`, `DEFINES=` (`-G`, `+define+`)	`TEST=`, `SEED=`, `PLUSARGS=`
Changing it needs	A new compilation (separate build directory)	Only a new run
Typical use	`WIDTH`, `DEPTH`, `MUL_XLEN`	Test name, seed, number of transactions, back-pressure rate, verbosity

The testbench top declares the design parameters and passes them to the DUT, so one `PARAMS` override changes both:

```
module tb_top #(
  parameter int WIDTH = 8,
  parameter int DEPTH = 16
);
  sync_fifo #(.WIDTH(WIDTH), .DEPTH(DEPTH)) dut (.*);
  initial $display("PARAMS: WIDTH=%0d DEPTH=%0d", WIDTH, DEPTH);
  // ...
endmodule
```

Runtime settings are read with `$test$plusargs` and `$value$plusargs`. Every setting has a default and is printed at the start of the log, so a log is self-describing:

```
string       test_name = "random";
int unsigned num_txn   = 10000;
int unsigned bp_pct    = 20;     // back-pressure probability in percent
int unsigned idle_pct  = 10;     // idle-cycle probability in percent
bit          verbose   = 0;

function automatic void read_config();
  void'($value$plusargs("TEST=%s",     test_name));
  void'($value$plusargs("NUM_TXN=%d",  num_txn));
  void'($value$plusargs("BP_PCT=%d",   bp_pct));
  void'($value$plusargs("IDLE_PCT=%d", idle_pct));
  if ($test$plusargs("VERBOSE")) verbose = 1;
  $display("CONFIG: TEST=%s SEED=%0d NUM_TXN=%0d BP_PCT=%0d IDLE_PCT=%0d",
           test_name, get_seed(), num_txn, bp_pct, idle_pct);
endfunction
```

An unknown `+TEST=` name is an error that ends in `TEST FAILED`, never a silent fall-back to a default test: a typo in a regression list would otherwise run the wrong test forever and report success.

TEST TERM | Random test

Stimulus drawn from `$urandom` with a printed seed, thousands of transactions, checked by a reference model and a scoreboard. It finds the bugs nobody thought of; a pass proves correctness only for the stimulus actually generated, which is why cover properties and many seeds are needed. The seed makes any failure reproducible.

4.2 Randomization Functions

Function	Returns	Notes
`$random`	Signed 32-bit	Verilog legacy. Can be negative. Seed is an optional `inout` variable.
`$urandom`	Unsigned 32-bit	Preferred. Each process (thread) has its own random generator, so adding a thread elsewhere does not change this thread's sequence.
`$urandom_range(max, min)`	Unsigned in $[min, max]$	Both limits inclusive; `min` defaults to 0; if `min > max`, the limits are swapped.
`std::randomize(v) with {...}`	Success flag	Constrained randomization without classes. Check simulator support before using it; Lab V3 covers constraints properly.

Common mistakes to avoid:

```
int x;
x = $random % 10;            // BAD: range is -9..9, not 0..9
```

```
x = $urandom % 10;           // OK, but $urandom_range says what you mean
x = $urandom_range(9);       // GOOD: 0..9
x = $urandom_range(10, 1);   // GOOD: 1..10
logic [63:0] w = $urandom;   // BAD: upper 32 bits are always 0
logic [63:0] v = {$urandom, $urandom};   // GOOD

// Weighted stimulus: 70% push, 20% pop, 10% both
case ($urandom_range(99)) inside
  [0:69]:  begin push_r = 1; pop_r = 0; end
  [70:89]: begin push_r = 0; pop_r = 1; end
  default: begin push_r = 1; pop_r = 1; end
endcase
```

Purely uniform data almost never hits the values that break datapaths: the probability of drawing `0` for a 32-bit operand is 2^{-32}. Random tests must therefore mix uniform data with corner values using weights:

```
function automatic logic [XLEN-1:0] rand_operand();
  case ($urandom_range(99)) inside
    [0:59]:  return XLEN'({$urandom, $urandom});   // uniform
    [60:69]: return '0;
    [70:79]: return '1;                            // all ones
    [80:84]: return XLEN'(1);
    [85:89]: return {1'b1, {(XLEN-1){1'b0}}};      // most negative
    [90:94]: return {(XLEN/2){2'b01}};             // alternating
    default: return XLEN'(1) << $urandom_range(XLEN-1);  // power of two
  endcase
endfunction
```

4.3 Reproducibility

A random test that fails once and cannot be repeated is worse than no test: it proves a bug exists and gives you no way to find it.

- The Makefile passes the same `SEED` twice: as `+verilator+seed+$(SEED)`, which seeds `$urandom` in Verilator, and as `+SEED=$(SEED)`, which the testbench reads and prints. When `SEED` is not given, the Makefile picks a new one (for example from `date +%s`) and the testbench prints it.
- Running the same command twice must give identical logs. Check it: `diff` the two logs of the same seed.
- Everything random comes from `$urandom` and `$urandom_range` in the testbench. Do not use `$random`, the wall clock, or the order of hash iteration. Use `$random` once, in a small exercise, and explain in `doc/verification.md` why it was replaced.
- Because `$urandom` is per-thread, adding a new background process (for example a second back-pressure generator) does not change the sequence of the existing ones. Starting threads in a different order can; keep the start order fixed.
- Verilator is two-state, so an uninitialized flip-flop starts at 0 and can hide a missing reset (rules 6 and 7 of Lab R1). In this lab, extend `flow/mk/sim_verilator.mk` to build with `--x-assign unique --x-initial unique` and to run with `+verilator+rand+reset+2`, so uninitialized state gets random values derived from the same seed. A result that changes between seeds for a *directed* test points at missing reset.

4.4 Shared Utilities: `tb_utils_pkg`

Every testbench of the program ends with exactly one `TEST PASSED` or `TEST FAILED` line, has a watchdog and prints its configuration (L0 conventions). Write that code once:

```
// dv/common/tb_utils_pkg.sv
package tb_utils_pkg;
  int unsigned n_checks = 0, n_errors = 0;
  int unsigned max_errors = 20;        // stop early, keep logs readable

  function automatic int unsigned get_seed();
    int unsigned s = 0;
    void'($value$plusargs("SEED=%d", s));
    return s;
  endfunction

  function automatic int unsigned get_int_arg(string name, int unsigned dflt);
    int unsigned v = dflt;
    void'($value$plusargs({name, "=%d"}, v));
    return v;
  endfunction

  function automatic void check(bit ok, string msg);
    n_checks++;
    if (!ok) begin
      n_errors++;
      $display("ERROR: [%0t] CHECK FAILED: %s", $time, msg);
      if (n_errors >= max_errors) begin report(); $finish; end
    end
  endfunction

  function automatic void report();
    $display("REPORT: checks=%0d errors=%0d", n_checks, n_errors);
    // a test that checked nothing has not passed
    if (n_errors == 0 && n_checks > 0) $display("TEST PASSED");
    else                               $display("TEST FAILED");
  endfunction

  // action block of every assertion: else tb_utils_pkg::sva_fail("a_x");
  function automatic void sva_fail(string label);
    check(1'b0, {"ASSERTION FAILED: ", label});
  endfunction

  task automatic watchdog(time limit);
    #(limit);
    $display("ERROR: WATCHDOG: no end of test after %0t", limit);
    n_errors++;
    report();
    $finish;
  endtask
endpackage
```

Three details are deliberate. A test with zero checks fails, because a broken monitor that never fires would otherwise produce a perfect pass. Errors are printed with `$display`, not `$error`: Verilator stops the simulation at the first `$error`, so the scoreboard could not count and the final verdict line would never be printed. And assertion failures must make the test fail in every simulator: the default action of a failing assertion is an `$error`, which stops Verilator but lets VCS continue to `TEST PASSED`. Every assertion therefore gets an explicit action block, `else tb_utils_pkg::sva_fail("a_label")`, that counts the failure. A reference version of the package can also count cover-property hits

(`cover_hit(label)` as the pass action) and print them in `report()` as `COVER: <label> <hits>` lines, which `make regress` sums over all runs.

5 Required Tests

5.1 `seq_mul`

`+TEST=`	Scenario
`smoke`	The directed cases of the Lab R1 smoke test, now through the driver and scoreboard
`corners`	$0 \times x$, $1 \times x$, all-ones × all-ones, the most negative value (`'h8000_0000`) in each operand, powers of two — for every implemented operation
`back_to_back`	A new request in the cycle after each result is accepted, `out_ready` always 1
`hold`	`out_ready` held low for many cycles (`+BP_PCT=90`); `out_result` must stay stable
`busy_valid`	`in_valid` asserted continuously while the unit is busy; no second request may be accepted
`random`	At least 10,000 requests (`+NUM_TXN=`), random operations, operands from `rand_operand()`, random idle cycles and back-pressure
`exhaustive`	With `DEFINES=MUL_XLEN=8`: every operand pair for every implemented operation, with random back-pressure

MULH and MULHSU are tested if you implemented them in Lab R1 (stretch); otherwise the tests skip them and the testplan says so.

5.2 `sync_fifo`

`+TEST=`	Scenario
`smoke`	Few pushes and pops, as in the Lab R2 smoke test
`fill_drain`	Fill to full, push while full, drain to empty, pop while empty
`simultaneous`	Push and pop in the same cycle at empty, mid-level and full
`random`	Weighted random push/pop (`+PUSH_PCT=`, `+POP_PCT=`), data mixing uniform and corner values, at least 10,000 cycles

Every test must pass for the parameter sets of the regression, including a `DEPTH` that is not a power of two and the minimum `DEPTH=2`.

6 Regression

TEST TERM | Regression

> The whole set of tests × seeds × parameter sets, run by one command with a pass/fail summary and a non-zero exit code on any failure. It is run before every merge; a *nightly regression* runs fresh seeds every night. A pass proves that nothing that used to work is broken — for the stimulus and configurations in the list.

One passing run proves little. A **regression** runs every test with several seeds on every relevant configuration and reports one line per run. It is what runs in CI after every merge and every night on a real project; it is also how you prove that a bug fix did not break something else.

6.1 The Regression List

Each IP has a `dv/regress.list`. `config` lines name a compile-time configuration (make variable assignments); `test` lines give a test, the number of seeds, the configurations it runs on (`*` = all) and optional plusargs:

```
# hw/ip/sync_fifo/dv/regress.list
# config <name> <make variables>
config default
config w32d5    PARAMS="WIDTH=32 DEPTH=5"
config w1d2     PARAMS="WIDTH=1 DEPTH=2"
# test <name> <seeds> <configs> [plusargs]
test smoke         1  *
test fill_drain    3  *
test simultaneous  3  *
test random        5  *        +NUM_TXN=10000 +BP_PCT=30
```

```
# hw/ip/seq_mul/dv/regress.list
config default
config xlen8    DEFINES=MUL_XLEN=8
test smoke         1  *
test corners       1  *
test back_to_back  3  *
test hold          3  *        +BP_PCT=90
test busy_valid    3  *
test random        5  default  +NUM_TXN=10000
test exhaustive    1  xlen8
```

Seeds are `1..N`, so a regression is itself reproducible. A nightly regression on a real project uses fresh seeds every night and records them; you may add that as an option.

6.2 `flow/mk/regress.mk`

The `regress` target is design-independent and calls the existing `sim` target for each combination, so there is exactly one way to run a test. A starting point:

```
# flow/mk/regress.mk -- `make regress` (Lab V1)
REGRESS_LIST ?= dv/regress.list
REGRESS_SUM  := $(REPO_ROOT)/build/$(IP)/regress_summary.txt

.PHONY: regress
regress:
        @mkdir -p $(dir $(REGRESS_SUM)); rm -f $(REGRESS_SUM); \
        grep '^config' $(REGRESS_LIST) | while read -r _ cfg vars; do \
          grep '^test' $(REGRESS_LIST) | while read -r _ t n cfgs pa; do \
            case ",$$cfgs," in *",$$cfg,"*|",*,") ;; *) continue ;; esac; \
            for s in $$(seq 1 $$n); do \
              if eval $(MAKE) --no-print-directory sim $$vars TEST=$$t \
                   SEED=$$s PLUSARGS=\"$$pa\" </dev/null >/dev/null 2>&1; \
              then r=PASS; else r=FAIL; fi; \
              printf '%-10s %-14s %5s  %s\n' $$cfg $$t $$s $$r \
                | tee -a $(REGRESS_SUM); \
            done; \
          done; \
        done; \
        echo "TOTAL: $$(grep -c . $(REGRESS_SUM)) runs," \
             "$$(grep -c FAIL $(REGRESS_SUM)) failed"; \
        ! grep -q FAIL $(REGRESS_SUM)
```

A failing run must be re-runnable from its summary line alone; the reference implementation prints a ready-to-paste `rerun: make ... sim ...` line for each failure, builds every configuration once before its runs, and supports `REGRESS_CONFIGS`/`REGRESS_TESTS` subsets, `REGRESS_SEED_BASE` (nightly) and `REGRESS_JOBS` (parallel runs).

Requirements:

- Each run keeps its normal log, `build/<ip>/<cfg>/sim/<test>_s<seed>.log`; the summary table goes to `build/<ip>/regress_summary.txt`. A failing line must be enough to re-run that exact case with `make sim`.
- Each configuration is compiled once, in its own build directory (L0 rule: a change of `PARAMS` or `DEFINES` never reuses a model built with other values).
- `make regress` exits non-zero if any run failed, so CI can rely on it.
- Nothing in `regress.mk` names a design. Lab V2 uses it unchanged for `arb_mem`.

Running the runs in parallel (`make -j`) is a worthwhile stretch; note that runs of the same configuration share a build directory, so build each configuration first.

7 Assertions (SVA)

TEST TERM | Assertion (SVA) checks

> Properties checked on every clock edge during simulation (or proven by a formal tool). They catch a protocol or design-rule violation at the cycle and on the signal where it happens, instead of when a wrong value finally reaches the scoreboard. A passing assertion proves nothing if its condition never occurred — hence the cover properties.

An assertion is an executable statement of something that must always be true. A scoreboard finds a wrong result at the output, often many cycles after the cause; an assertion fires at the cycle and the signal where the rule is broken. Designers write assertions for the design's internal assumptions; verification engineers write them for interface protocols and for the specification.

7.1 Immediate and Concurrent Assertions

```
// Immediate: checked when the statement executes, like an if
always_comb begin
  assert (!(full && empty)) else $error("full and empty at the same time");
end

// Concurrent: checked on every clock edge, can describe behavior over time
a_not_full_empty: assert property (@(posedge clk) disable iff (!rst_n)
  !(full && empty));
```

Immediate assertions can fire on short glitches while combinational logic settles. Prefer concurrent assertions for anything clocked. Always give an assertion a label (`a_not_full_empty:`); the label appears in the failure message and in the testplan.

7.2 Sampling: Why Assertions Do Not Race

Concurrent assertions use the values sampled in the **Preponed** region (Lab R2): the values just before the clock edge, before any process of that time step has run. The assertion is then evaluated

in the **Observed** region, and its action block runs in the **Reactive** region. This is why an assertion sees the same values a flip-flop samples, and why it is free of the races described in Lab R2.

7.3 Implication and Delay

Operator	Meaning
`a |-> b`	If `a` holds at this edge, `b` must hold at the *same* edge (overlapping)
`a |=> b`	If `a` holds at this edge, `b` must hold at the *next* edge (non-overlapping)
`##n`	Exactly `n` clock cycles later
`##[m:n]`	Between `m` and `n` cycles later
`##[1:$]`	Eventually (one or more cycles later, no upper bound)
`a[*n]`	`a` holds for `n` consecutive cycles
`disable iff (!rst_n)`	Do not check (and abort pending checks) during reset

If the condition on the left of an implication is false, the assertion passes vacuously. A property that only ever passes vacuously checks nothing; this is why cover properties (below) are required.

7.4 Sampled-Value Functions

These functions compare the sampled value at the current clock edge with the sampled value at the previous clock edge.

Function	True when
`$rose(x)`	The least significant bit of `x` changed to 1 since the previous edge (0, X or Z → 1)
`$fell(x)`	The least significant bit of `x` changed to 0 since the previous edge
`$stable(x)`	`x` has the same value as at the previous edge
`$changed(x)`	`x` has a different value than at the previous edge
`$past(x, n)`	Returns the value of `x` `n` edges ago (default 1)
`$isunknown(x)`	`x` contains X or Z (always false in a two-state simulator)
`$onehot(x)`, `$onehot0(x)`	Exactly one bit set; at most one bit set
`$countones(x)`	Number of bits set to 1

`$rose` and `$fell` do not detect a clock-edge event; they describe a change between two consecutive samples. A pulse that rises and falls between two clock edges is invisible to them, exactly as it is to a flip-flop.

```
// full can only become 1 after a push
a_full_after_push: assert property (@(posedge clk) disable iff (!rst_n)
  $rose(full) |-> $past(push));

// a push without pop, when not full, increases count by one
a_count_inc: assert property (@(posedge clk) disable iff (!rst_n)
  push && !pop && !full |=> count == $past(count) + 1);

// seq_mul: an accepted request produces a result within XLEN+2 cycles
a_latency: assert property (@(posedge clk) disable iff (!rst_n)
  in_valid && in_ready |-> ##[1:XLEN+2] out_valid);

// seq_mul: the result is held while the consumer is not ready
a_out_hold: assert property (@(posedge clk) disable iff (!rst_n)
```

```
  out_valid && !out_ready |=> out_valid && $stable(out_result));
```

7.5 Assert, Assume and Cover

- `assert property`: the design must satisfy it.
- `assume property`: a rule the environment (inputs) must follow. In simulation it is checked like an assertion — so it checks your *testbench*; in formal verification it constrains the inputs.
- `cover property`: reports that a scenario actually happened. Coverage proves that your tests reached the interesting cases and that your assertions were not only passing vacuously.

7.6 Where Assertions Go

- **Design assertions** (internal invariants, written by the designer, for example "the write pointer is always below `DEPTH`") go inside the RTL module, guarded so synthesis never sees them. From this lab on, the repository rule is: RTL contains assertions only inside `` `ifndef SYNTHESIS ``.

```
`ifndef SYNTHESIS
  a_wptr_range: assert property (@(posedge clk) disable iff (!rst_n)
    wr_ptr < DEPTH);
`endif
```

- **Specification and interface checks** (written by verification) go in a separate checker module in `dv/sva/<ip>_sva.sv`, attached with `bind` from `dv/sva/<ip>_bind.sv`. `bind` instantiates the checker *inside* every instance of the target module without editing the RTL, so the checker can also see internal signals by name, and it disappears from synthesis because the bind file is only in `dv.f`.

```
// dv/sva/sync_fifo_sva.sv
module sync_fifo_sva #(parameter int WIDTH = 8, parameter int DEPTH = 16) (
  input logic                         clk, rst_n,
  input logic                         push, pop, full, empty,
  input logic [WIDTH-1:0]             push_data, pop_data,
  input logic [$clog2(DEPTH+1)-1:0] count
);
  default clocking @(posedge clk); endclocking
  default disable iff (!rst_n);

  a_not_full_empty: assert property (!(full && empty))
    else tb_utils_pkg::sva_fail("a_not_full_empty");
  a_full_def:       assert property (full  == (count == DEPTH))
    else tb_utils_pkg::sva_fail("a_full_def");
  // ... the other required assertions and cover properties
endmodule

// dv/sva/sync_fifo_bind.sv
bind sync_fifo sync_fifo_sva #(.WIDTH(WIDTH), .DEPTH(DEPTH)) u_sva (.*);
```

`.*` connects every checker port to the signal with the same name in the scope of `sync_fifo`; use explicit connections for internal signals and when the RTL uses different names, for example `bind sync_fifo sync_fifo_sva #(...) u_sva (.*, .rd_ptr_q(rd_ptr_q));`. Figure 2 shows what the bind creates.

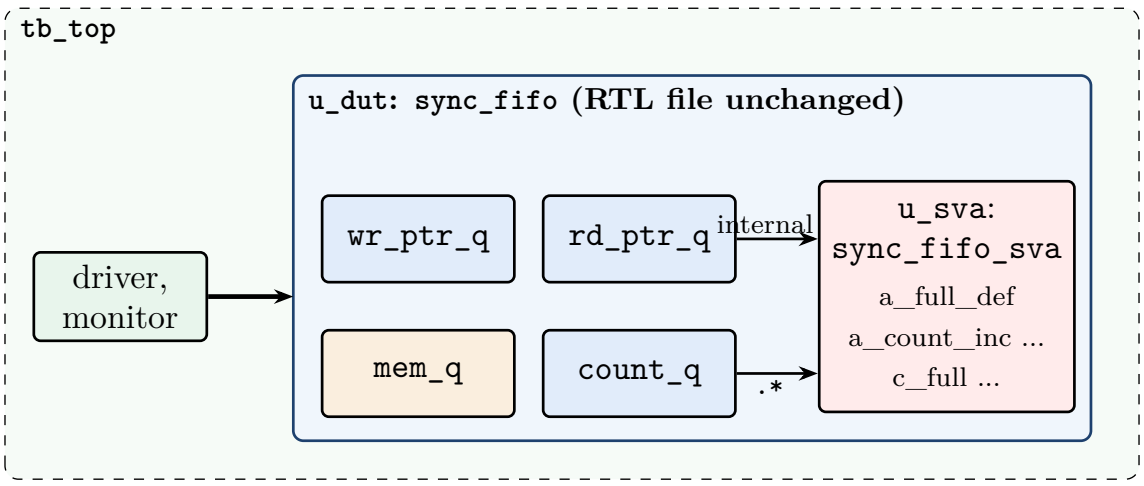


Figure 2: What `bind` does. `dv/sva/sync_fifo_bind.sv` contains `bind sync_fifo sync_fifo_sva #(.WIDTH(WIDTH), .DEPTH(DEPTH)) u_sva (.*, ...);` and is only in `dv/dv.f`, so synthesis never sees it. The checker is instantiated *inside* every `sync_fifo` instance (hierarchical name `tb_top.u_dut.u_sva`), so it can read ports and internal signals by name, while the RTL file is not edited.

The Makefile passes `--assert` to Verilator for simulation — without it Verilator 5.013 drops every assertion silently (a failing property passes); Lab P1 passes `-DSYNTHESIS` to sv2v.

Sometimes a property needs state that the design does not provide. Write that state as *auxiliary code* in the checker module, not in the RTL. For example, "no new request is accepted while busy" is easiest with a flag:

```
// in seq_mul_sva: busy from acceptance until the result is taken
logic busy;
always_ff @(posedge clk or negedge rst_n)
  if (!rst_n)                       busy <= 1'b0;
  else if (in_valid && in_ready)    busy <= 1'b1;
  else if (out_valid && out_ready)  busy <= 1'b0;

a_no_accept_busy: assert property (busy |-> !in_ready);
```

NOTE | Tool note.

Verilator 5 supports a growing subset of SVA: immediate and concurrent assertions, `|->`, `|=>`, the sampled-value functions and `cover property` are supported; ranges and sequence operators depend on the version. Verilator 5.013 (installed on the team cloud) rejects `default disable iff`, `##[m:n]` and `##[1:$]`, `##1` inside a sequence and `[*n]` ("Unsupported"): write `disable iff (!rst_n)` in every property, and rewrite the others with auxiliary code (a counter or a flag, as above), for example under `` `ifdef VERILATOR ``, and record the rewrite in `doc/verification.md`. Cover-property pass actions only run when the model is built with `--coverage-user` (the flow adds it). Two more 5.013 traps: a comment that starts with the word "verilator" is parsed as a pragma, and a cover action that increments a module variable crashed the compiler (call a package function instead). Never silently delete an assertion the tool rejects.

SYNOPSYS CLOUD

Run this lab with VCS: `make sim SIM=vcs` and `make regress SIM=vcs` (the cloud default). VCS supports the full SVA language used here, including `##[1:$]`, `[*3]` and `default disable iff`, is four-state (`$isunknown` and X results become visible), and prints a per-property summary ("*N attempts, M match*") of every cover property at the end of the log. Debug an assertion failure in Verdi: `make sim WAVES=1 TEST=... SEED=...` then `make waves` (FSDB; Verdi shows assertion results on the waveform). `make sim COV=1` + `make cov` gives code and assertion coverage with URG (used in Lab V3). To *prove* the assertions instead of simulating them, VC Formal (`vcf`) reads the same checker and bind files; the `assume`

`property` statements become input constraints. Note: on the cloud's Verilator 5.013 the V1 environment does not run correctly — its `--timing` scheduler lets the monitor tasks woken by `@(posedge clk)` see DUT outputs *after* that edge's updates (not IEEE behavior), so a monitor misses a handshake. Use VCS for V1; the R1/R2 smoke testbenches run on both.

7.7 Required Assertions

`sync_fifo` (at least the following):

1. `full` and `empty` are never both true.
2. `full == (count == DEPTH)` and `empty == (count == 0)`.
3. `count` changes by exactly +1, −1 or 0 according to the accepted push and pop of the previous cycle (use `$past`).
4. `$rose(full)` only after a push; `$fell(empty)` only after a push.
5. `pop_data` is stable while `!empty` and there was no pop (use `$stable`).
6. `count` never exceeds `DEPTH`.

`seq_mul` (at least the following):

1. `out_result` stable and `out_valid` held while `out_valid && !out_ready`.
2. `out_valid` and `in_ready` never X after reset (`$isunknown`; effective only in four-state simulators, see the review questions).
3. No new request accepted while busy (auxiliary flag above).
4. An accepted request produces a result within `XLEN+2` cycles (the latency bound).
5. In idle after reset: `in_ready` is 1 and `out_valid` is 0.

Cover properties (at least): FIFO becomes full; FIFO goes from full to empty; push and pop in the same cycle while full; push and pop in the same cycle while empty; read pointer wraps; `seq_mul` result held for at least 3 cycles (`out_valid && !out_ready [*3]`); `seq_mul` back-to-back requests (a request accepted in the cycle after a result was taken).

Every assertion and cover property appears in `dv/testplan.md`, and the regression shows that every cover property was hit at least once.

7.8 Exercise: See an Assertion Fire First

TEST TERM | Negative / error-injection test (bug hunt)

A deliberately broken design (or illegal stimulus) is run to prove that the checkers — scoreboard and assertions — actually fire. A checker that has never failed proves nothing: this test is passed when the environment *fails*, with a message that points at the bug.

Reading about assertions is not enough; you must see one point at a bug. Use the buggy RTL versions provided by the coordinator, or insert bugs yourself in a throw-away branch (for example an off-by-one in the FIFO pointer wrap for odd `DEPTH`, `full` computed one cycle late, `out_valid` dropped without `out_ready`, the accumulator not cleared between two requests).

For at least one bug per design, record in `doc/verification.md`:

1. The bug, the failing test and the seed.

2. The first assertion failure (time, label) and the first scoreboard error (time), and how many cycles apart they are.
3. How you found the root cause starting from each message, and how much faster the assertion got you there.
4. Whether any bug was *not* caught, and which test or assertion you added so that it is.

NOTE | If you find a real bug.

The V labs verify the R designs as they are. If a test fails because the RTL is wrong (not the testbench), fix the RTL in `hw/ip/<ip>/rtl/` with a separate commit whose message names the failing test, configuration and seed (for example `fix(sync_fifo): rd_ptr wrap for DEPTH=5, found by random s3 w32d5`), add the case to the bug log in `doc/verification.md`, and make sure the smoke test of Phase 1 still passes.

8 Requirements / Tasks

1. Write `dv/testplan.md` for both IPs from their `doc/spec.md` before writing tests.
2. Write `dv/common/tb_utils_pkg.sv` with at least `check`, `report`, `watchdog`, `get_seed` and `get_int_arg`.
3. Build the `seq_mul` environment in `hw/ip/seq_mul/dv/`: driver, input and output monitors, reference model, scoreboard, back-pressure, tests of Section 5.1; all inputs driven with `<=`.
4. Build the `sync_fifo` environment in `hw/ip/sync_fifo/dv/` with a queue-based reference model and the tests of Section 5.2.
5. Read every runtime setting with plusargs (`TEST`, `NUM_TXN`, `BP_PCT`, `IDLE_PCT`, and the FIFO push/pop weights) with defaults printed in a `CONFIG:` line; reject unknown test names.
6. Use only `$urandom`/`$urandom_range` for stimulus, mix corner values with weights, and show that two runs with the same seed produce identical logs.
7. Extend `flow/mk/sim_verilator.mk` with random initialization driven by `SEED`; write `flow/mk/regress.mk` and a `dv/regress.list` per IP; `make regress` passes for both IPs (FIFO: at least 3 parameter sets including `DEPTH=2` and an odd `DEPTH`; `seq_mul`: default and `MUL_XLEN=8`).
8. Write the required assertions and cover properties in `dv/sva/` and bind them; every cover property is hit in the regression.
9. Complete the assertion-fires-first exercise for both designs.
10. Keep `make check-all` green: the Phase 1 smoke tests still pass, and `make lint` of the RTL is unchanged.

9 Repository and Flow

Files created or changed in this lab:

```
dv/common/tb_utils_pkg.sv                    new, shared by all V labs
flow/mk/regress.mk                           new: `regress`
flow/mk/sim_verilator.mk                     changed: --assert, random init from SEED
hw/ip/seq_mul/Makefile                       TB_FLIST=dv/dv.f, TB_TOP=tb_top, include
    regress.mk
hw/ip/seq_mul/dv/{testplan.md,regress.list,dv.f}
hw/ip/seq_mul/dv/tb/tb_top.sv
hw/ip/seq_mul/dv/env/seq_mul_{driver,monitor,scoreboard}.svh
```

```
hw/ip/seq_mul/dv/tests/seq_mul_tests.svh
hw/ip/seq_mul/dv/sva/seq_mul_{sva,bind}.sv
hw/ip/seq_mul/doc/verification.md            new: results, bug log
hw/ip/sync_fifo/...                          same set of files
hw/ip/<ip>/rtl/*                             only for bug fixes or `ifndef SYNTHESIS
    assertions
```

The testbench file list:

```
// hw/ip/sync_fifo/dv/dv.f  (the flow compiles only TB_FLIST: include the RTL list)
-f ${REPO_ROOT}/hw/ip/sync_fifo/rtl/sync_fifo.f
+incdir+${REPO_ROOT}/hw/ip/sync_fifo/dv/env
+incdir+${REPO_ROOT}/hw/ip/sync_fifo/dv/tests
${REPO_ROOT}/dv/common/tb_utils_pkg.sv
${REPO_ROOT}/hw/ip/sync_fifo/dv/sva/sync_fifo_sva.sv
${REPO_ROOT}/hw/ip/sync_fifo/dv/sva/sync_fifo_bind.sv
${REPO_ROOT}/hw/ip/sync_fifo/dv/tb/tb_top.sv
```

The IP Makefile switches its default testbench to the V environment; the smoke test remains reachable by overriding the two variables:

```
# hw/ip/sync_fifo/Makefile (changes)
TB_TOP    ?= tb_top
TB_FLIST  ?= dv/dv.f
include $(REPO_ROOT)/flow/mk/regress.mk
```

Example commands that must work:

```
make -C hw/ip/sync_fifo sim TEST=random SEED=7 PLUSARGS="+NUM_TXN=20000 +BP_PCT=50"
make -C hw/ip/sync_fifo sim PARAMS="WIDTH=32 DEPTH=5" TEST=fill_drain WAVES=1
make -C hw/ip/sync_fifo sim TB_TOP=tb_sync_fifo TB_FLIST=tb/tb.f   # R2 smoke
make -C hw/ip/seq_mul   sim DEFINES=MUL_XLEN=8 TEST=exhaustive
make -C hw/ip/seq_mul   regress
make regress IP=sync_fifo                    # from the top level
make check-all
```

10 Deliverables

One merge request from branch `lab/v1-selfcheck-sva`, tagged `v1-done` after review, containing all files of the previous section. Per IP:

- `dv/testplan.md`: every feature with its tests, assertions and cover items; gaps marked.
- `doc/verification.md`:
  - environment structure (components and files) and the list of tests and plusargs, with an example command for each;
  - the output of `make regress`, all passing (FIFO: at least 3 parameter sets × all tests × the seeds in the list);
  - the assertion and cover lists, with the cover counts showing every cover property was hit;
  - the reproducibility check (same seed, identical logs) and the `$random` versus `$urandom` note;
  - the assertion-fires-first exercise;
  - the bug log: every real RTL bug found, with the failing test, seed and fix commit ("none found" is a valid entry).

- `README.md`: updated with the new `make` targets and the verification status.

11 Assessment

Criterion	Weight
Environment structure: driver, monitors, reference model, scoreboard, `<=` driving	20%
Tests: corner, handshake, random (≥10k), exhaustive; all passing	20%
Assertions and cover properties, bound from `dv/sva/`	20%
Randomization, plusargs and reproducibility	10%
Regression: `regress.mk`, `regress.list`, `tb_utils_pkg`	15%
Testplan and assertion-fires-first exercise	10%
Documentation (`verification.md`, README)	5%
Total	100%

12 Review Questions

The coordinator will ask questions like these in the review. Be ready to answer them without notes.

1. Why does your monitor identify a transaction by the handshake and not by counting cycles? Which later lab depends on that?
2. Your driver drives with `<=` right after a rising edge and reads `in_ready` at the next edge. Which value of `in_ready` does it see, and why is that the value the DUT used?
3. Why can't a plusarg change `DEPTH`? What happens to your regression if `PARAMS` changes but the build directory is reused?
4. A random test failed last night. What exactly do you need to reproduce it, and where in the log is it?
5. Why is `$random % 10` wrong, and why does adding a new thread not change the stimulus of the existing ones with `$urandom`?
6. What is the difference between `|->` and `|=>`? Rewrite one of your assertions using the other operator.
7. Which of your assertions can pass vacuously, and which cover property proves that it did not?
8. `$rose(full)` is true at an edge. What exactly did the assertion compare?
9. Why does `$isunknown` never fire in your Verilator runs, and when would it?
10. In the exercise, how many cycles before the scoreboard did the assertion fire, and why can the gap be very large for a FIFO?

13 References

- IEEE Std 1800-2017, *SystemVerilog Language Reference Manual*: Clause 4 (scheduling semantics), Clause 16 (assertions), Clause 18 (random number generation), Clause 21 (plusargs), Clause 23.11 (`bind`).

- Verilator User's Guide: `--assert`, `--coverage-user`, `--x-assign`, `--x-initial`, runtime arguments `+verilator+seed+` and `+verilator+rand+reset+`.
- OpenTitan, Design Verification Methodology (testplans, `dv/` layout, regressions with `dvsim`).
- Clifford E. Cummings, Nonblocking Assignments in Verilog Synthesis, Coding Styles That Kill!, SNUG 2000.
- Verification Academy: SystemVerilog Assertions and testbench material.
- *SystemVerilog Assertions Handbook*, Cohen, Venkataramanan, Kumari and Piper.